

## INFORME DE CIERRE DEL PROYECTO

# Conecta +

Plataforma de gestión de donaciones de alimentos y recursos entre empresas y organizaciones sociales

---

**Fases:** Implementación y seguridad · Pruebas y calidad · Cierre y evaluación

**Repositorio:** <https://github.com/EvanDavidMS/conecta-mas-donaciones>

**Fecha:** 28 de septiembre de 2026

Contenido

1. Resumen ejecutivo..... 3

2. Alcance y solución implementada ..... 3

    2.1 Roles del sistema ..... 3

    2.2 Ruta guiada por rol ..... 3

    2.3 Flujo de una donación..... 4

    2.4 Arquitectura y decisiones técnicas ..... 4

3. Comparación entre lo planificado y lo ejecutado ..... 7

    3.1 Alcance planificado frente al entregado ..... 8

    3.2 Incidentes que afectaron el cronograma ..... 8

4. Resultados de pruebas y calidad..... 9

    4.1 Pruebas unitarias y de integración (Jest + Supertest)..... 9

    4.2 Pruebas de seguridad (OWASP ZAP)..... 9

    4.3 Análisis de calidad del código (SonarQube)..... 10

5. Lecciones aprendidas ..... 11

    5.1 Durante la planificación ..... 11

    5.2 Durante la implementación ..... 11

6. Plan de mejora continua ..... 11

    6.1 Propuestas de innovación..... 12

    6.2 Seguimiento ..... 12

7. Conclusiones ..... 12

## 1. Resumen ejecutivo

Conecta + es un sistema web que conecta a empresas o personas que tienen excedentes de alimentos y recursos (donadores) con organizaciones sociales que los necesitan (beneficiarias). Un equipo administrador verifica a las organizaciones, asigna cada donación y da trazabilidad a todo el proceso.

El proyecto se completó en sus tres fases. Se entregó un sistema funcional con autenticación JWT y tres roles, un pipeline CI/CD que prueba, analiza, construye y despliega automáticamente en un entorno de prueba, y un ciclo completo de pruebas de seguridad y calidad con identificación y corrección de hallazgos.

| Indicador                               | Meta                        | Resultado                                                                      |
|-----------------------------------------|-----------------------------|--------------------------------------------------------------------------------|
| Pruebas automatizadas                   | —                           | 131 pruebas en 7 suites, 0 fallos                                              |
| Cobertura de pruebas unitarias          | ≥ 80 %                      | 99.84 % líneas · 96.89 % ramas · 99.34 % funciones                             |
| Etapas del pipeline CI/CD               | Pruebas, build y despliegue | 5 etapas: pruebas, SonarQube, build Docker, staging + smoke + ZAP, publicación |
| Vulnerabilidades OWASP ZAP (Alto/Medio) | 0                           | 0 / 0                                                                          |
| Quality Gate SonarQube                  | Aprobado                    | Aprobado · Seguridad A · Fiabilidad A · Mantenibilidad A                       |
| Esfuerzo                                | 9 h planificadas            | 12 h ejecutadas (+33 %)                                                        |

## 2. Alcance y solución implementada

### 2.1 Roles del sistema

| Rol           | Responsabilidades                                                                                                                                                                             |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Donador       | Publica donaciones (la fecha de caducidad es obligatoria para alimentos), consulta su estado, cancela las que siguen disponibles y ve su impacto: kilos entregados y organizaciones apoyadas. |
| Beneficiario  | Se registra como organización (queda pendiente de verificación), explora y filtra el catálogo, solicita donaciones, cancela solicitudes pendientes y confirma la recepción.                   |
| Administrador | Verifica, suspende o reactiva cuentas; aprueba, rechaza o revoca solicitudes; registra entregas; consulta métricas globales y la auditoría. No puede crearse desde el registro público.       |

### 2.2 Ruta guiada por rol

Cada rol tiene una ruta guiada que recorre sus paneles, resalta cada elemento y explica para qué sirve: 10 pasos para el donador, 10 para el beneficiario y 11 para el administrador. Arranca sola la primera vez que el usuario entra, se puede repetir con el botón «Ruta guiada» de la barra superior y funciona en escritorio, en móvil y con el teclado.

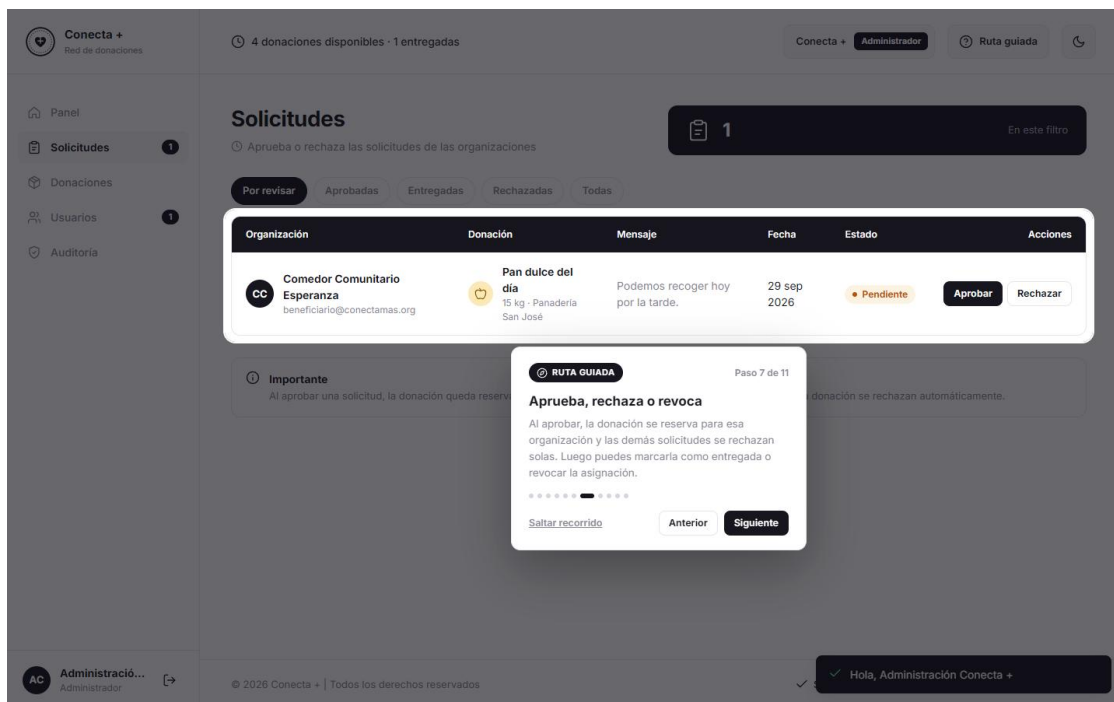


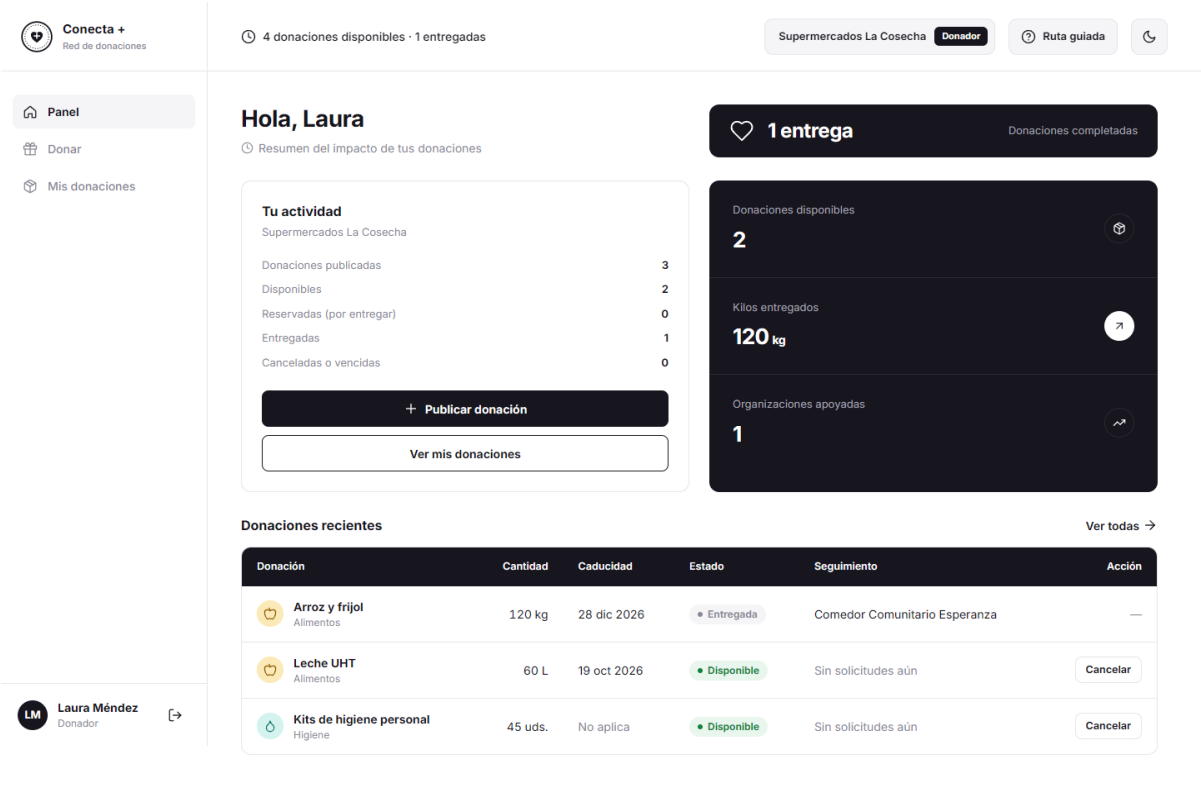
Figura 1. Ruta guiada del administrador: paso sobre el panel de solicitudes.

## 2.3 Flujo de una donación

Disponible → (una organización verificada la solicita) → el administrador aprueba y la donación queda Reservada; las demás solicitudes se rechazan automáticamente → la organización confirma la recepción y la donación queda Entregada. Si la fecha de caducidad llega sin asignación, la donación pasa a Vencida.

## 2.4 Arquitectura y decisiones técnicas

- **Backend:** Node.js 20 + Express, organizado en rutas, middleware y servicios de dominio.
- **Almacenamiento:** en el propio servidor (memoria + archivo JSON con escritura atómica). No requiere base de datos ni servicios de pago, como pidió el cliente.
- **Frontend:** HTML, CSS y JavaScript sin frameworks, basado en el diseño de referencia (monocromático, barra lateral, paneles oscuros de métricas), responsive y con modo oscuro.
- **Entrega:** imagen Docker multi-stage con usuario sin privilegios y healthcheck.





### 3. Comparación entre lo planificado y lo ejecutado

La tabla compara las horas planificadas por fase, según la guía de la actividad (4 h, 3 h y 2 h), con el esfuerzo real registrado para cada tarea y la causa de cada desviación.

| Fase                          | Tarea                                                       | Plan   | Real   | Desviación y causa                                                                                                                                          |
|-------------------------------|-------------------------------------------------------------|--------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Implementación y seguridad | Arquitectura y modelo de datos en servidor (sin BD)         | 0.5 h  | 0.5 h  | Sin desviación.                                                                                                                                             |
|                               | Registro y autenticación JWT con roles                      | 1 h    | 1.25 h | Se añadieron revocación de tokens en servidor y bloqueo por intentos fallidos (no planificados).                                                            |
|                               | Módulos de donaciones, solicitudes y administración         | 1 h    | 1.5 h  | El alcance pasó de 2 roles (admin/usuario) a 3 roles con verificación de organizaciones y máquina de estados.                                               |
|                               | Interfaz UI/UX responsive                                   | 0.5 h  | 1.25 h | Adaptación del diseño de referencia, tablas convertibles a tarjetas en móvil y modo oscuro.                                                                 |
|                               | Cambio solicitado: nombre «Conecta +» y ruta guiada por rol | 0 h    | 1 h    | No estaba en el plan: lo pidió el cliente al revisar la primera versión. Incluyó el motor del recorrido, 31 pasos en total y pruebas en escritorio y móvil. |
|                               | Pruebas unitarias ( $\geq 80\%$ )                           | 0.5 h  | 0.75 h | Se alcanzó 99.84 % de líneas con 131 pruebas.                                                                                                               |
|                               | Pipeline CI/CD (GitHub Actions)                             | 0.5 h  | 1 h    | La primera ejecución falló en el job de SonarQube (permisos y directorio de trabajo del scanner).                                                           |
| 2. Pruebas y calidad          | Escaneo OWASP ZAP (baseline + API activo autenticado)       | 1.5 h  | 1.25 h | Menor al plan: el escaneo se automatizó en un script reutilizable en local y en CI.                                                                         |
|                               | Corrección de hallazgos de ZAP                              | 0.5 h  | 0.25 h | Solo hubo 1 alerta Baja y avisos informativos.                                                                                                              |
|                               | Análisis SonarQube y métricas                               | 0.5 h  | 0.75 h | Rutas de lcof con “\” generadas en Windows y arranque lento de SonarQube ( $\approx 5$ min).                                                                |
|                               | Corrección de hallazgos de SonarQube                        | 0.5 h  | 0.5 h  | Credencial por defecto en el código y 8 code smells.                                                                                                        |
| 3. Cierre y evaluación        | Informe de cierre y lecciones aprendidas                    | 1.25 h | 1.25 h | Sin desviación.                                                                                                                                             |
|                               | Plan de mejora continua                                     | 0.75 h | 0.75 h | Sin desviación.                                                                                                                                             |

| Fase                          | Planificado | Ejecutado | Variación |
|-------------------------------|-------------|-----------|-----------|
| 1. Implementación y seguridad | 4 h         | 7.25 h    | +81 %     |
| 2. Pruebas y calidad          | 3 h         | 2.75 h    | -8 %      |
| 3. Cierre y evaluación        | 2 h         | 2 h       | 0 %       |
| Total                         | 9 h         | 12 h      | +33 %     |

### 3.1 Alcance planificado frente al entregado

| Requisito                    | Planificado                                   | Ejecutado                                                                                                                                             |
|------------------------------|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Usabilidad</b>            | No contemplada                                | Ruta guiada por rol en todos los paneles (cambio solicitado por el cliente)                                                                           |
| <b>Módulo básico</b>         | Registro de donantes                          | Registro de donadores y organizaciones, además de donaciones, solicitudes, verificación, entregas, métricas y auditoría                               |
| <b>Autenticación y roles</b> | JWT con roles administrador/usuario           | JWT HS256 con 3 roles, cookie httpOnly, revocación en servidor y bloqueo por intentos fallidos                                                        |
| <b>Pruebas unitarias</b>     | Cobertura $\geq 80$ % con Jest                | 131 pruebas; 99.84 % de líneas y 96.89 % de ramas; umbral del 80 % forzado en CI                                                                      |
| <b>CI/CD</b>                 | Despliegue automático en un entorno de prueba | GitHub Actions: pruebas $\rightarrow$ SonarQube $\rightarrow$ build Docker $\rightarrow$ staging + smoke test + ZAP $\rightarrow$ publicación en GHCR |
| <b>Seguridad</b>             | Escaneo OWASP ZAP (XSS, SQLi)                 | ZAP baseline con spider AJAX y escaneo activo autenticado de toda la API, dentro del pipeline                                                         |
| <b>Calidad</b>               | SonarQube con métricas documentadas           | SonarQube Community (contenedor efímero) con reporte automático antes y después de las correcciones                                                   |
| <b>Despliegue</b>            | Entorno de prueba                             | Staging efímero en el runner con Docker (sin costo). No se usaron servicios de pago                                                                   |

### 3.2 Incidentes que afectaron el cronograma

- **Límite de peticiones en las pruebas E2E:** el limitador anti fuerza bruta (20 intentos cada 15 min) bloqueó las pruebas automatizadas del navegador. Se resolvió con límites configurables por entorno. Además confirmó que el control funciona.
- **Diferencias entre Windows y Linux:** el archivo lcov generado en Windows usa “\” y SonarQube (Linux) no lo reconocía. Se normalizaron las rutas en el script de análisis.
- **Fallo del job de SonarQube en la primera ejecución del pipeline:** el scanner escribía su reporte en /tmp del contenedor y con otro UID. Se corrigió fijando sonar.working.directory, el usuario del contenedor y fetch-depth: 0 para el blame de git.
- **Revocación del token durante el escaneo:** el escaneo activo de ZAP llamaba a /logout, lo que invalidaba el JWT del propio escaneo. Se excluyó esa ruta del análisis.
- **Cambio de alcance a mitad del cierre:** tras revisar la primera versión, el cliente pidió renombrar el proyecto a «Conecta +» y añadir una ruta guiada. Se renombraron código, documentación, repositorio y la imagen Docker, y se repitieron los análisis de calidad y seguridad.
- **Entorno:** Docker Desktop estaba apagado y la CLI de GitHub no estaba instalada. Se arrancó el servicio y el repositorio se creó mediante la API REST de GitHub.

## 4. Resultados de pruebas y calidad

### 4.1 Pruebas unitarias y de integración (Jest + Supertest)

| Métrica    | Cubierto / total | %       |
|------------|------------------|---------|
| Sentencias | 713 / 714        | 99.85 % |
| Ramas      | 312 / 322        | 96.89 % |
| Funciones  | 152 / 153        | 99.34 % |
| Líneas     | 626 / 627        | 99.84 % |

Las pruebas cubren autenticación (tokens manipulados, alg=none, expirados, revocados), autorización por rol y por propietario (IDOR), validación de entradas (XSS, inyección, asignación masiva, escalada de rol), reglas de negocio completas y protecciones HTTP (CSP, CSRF, rate limiting, tamaño de petición y errores sin trazas). Se complementan con una prueba de humo de 17 verificaciones en staging y un recorrido E2E en navegador por los tres roles.

### 4.2 Pruebas de seguridad (OWASP ZAP)

Se ejecutaron dos escaneos contra el contenedor desplegado: un baseline (spider tradicional y AJAX con análisis pasivo) y un escaneo activo autenticado con JWT de administrador sobre todos los endpoints de la especificación OpenAPI, que incluye reglas de SQL Injection (genérica y time-based para MySQL, PostgreSQL, Oracle y MsSQL), XSS reflejado, persistente y DOM, inyección de comandos, SSTI, path traversal y CRLF.

| Riesgo      | Escaneo inicial | Tras las correcciones |
|-------------|-----------------|-----------------------|
| Alto        | 0               | 0                     |
| Medio       | 0               | 0                     |
| Bajo        | 1               | 1                     |
| Informativo | 8               | 8                     |

| Hallazgo                                                                                                         | Riesgo      | Acción                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unexpected Content-Type (rutas fuera de /api respondían HTML a métodos PATCH, p. ej. sondeos de metadatos cloud) | Bajo        | Corregido: fuera de la API solo se admiten GET/HEAD (405 JSON) y los clientes JSON reciben 404 en JSON. Quedan la raíz "/" (la aplicación HTML) y la página 404 para navegadores, comportamiento esperado. |
| Information Disclosure – Suspicious Comments (dom.js)                                                            | Informativo | Corregido: el comentario contenía "Todo...", que ZAP interpreta como TODO.                                                                                                                                 |
| Non-Storable / Non-Cacheable Content                                                                             | Informativo | Intencional: la API responde con Cache-Control: no-store porque maneja datos personales.                                                                                                                   |
| Information in Browser localStorage                                                                              | Informativo | Aceptado: solo se guardan la preferencia de tema y si ya se vio la ruta guiada. El token vive en una cookie httpOnly.                                                                                      |
| Authentication Request Identified / Client Error responses / Modern Web App                                      | Informativo | Esperado: identificación del login y respuestas 4xx de validación.                                                                                                                                         |

### 4.3 Análisis de calidad del código (SonarQube)

| Métrica                        | Análisis inicial | Tras las correcciones |
|--------------------------------|------------------|-----------------------|
| Quality Gate                   | Aprobado         | Aprobado              |
| Líneas de código               | 4031             | 4441                  |
| Bugs                           | 0                | 0                     |
| Vulnerabilidades               | 1                | 0                     |
| Security hotspots              | 0                | 0                     |
| Code smells                    | 8                | 0                     |
| Deuda técnica                  | 0 h 40 min       | 0 h 0 min             |
| Ratio de deuda técnica         | 0.0 %            | 0.0 %                 |
| Calificación de mantenibilidad | A                | A                     |
| Calificación de fiabilidad     | A                | A                     |
| Calificación de seguridad      | C                | A                     |
| Cobertura                      | 98.4 %           | 98.4 %                |
| Duplicación                    | 0.0 %            | 0.0 %                 |
| Complejidad cognitiva          | 371              | 406                   |

Correcciones aplicadas: (1) la vulnerabilidad S2068 se resolvió sacando las credenciales de demostración del código a un archivo de configuración (demo.env) y generando contraseñas aleatorias cuando no se definen; en producción siguen siendo obligatorias; (2) se eliminaron los operadores ternarios anidados (S3358); (3) se movió una función asíncrona al ámbito externo (S7721); (4) se usó encadenamiento opcional (S6582); (5) se reemplazó role="status" por el elemento <output> por accesibilidad (S6819).

## 5. Lecciones aprendidas

### 5.1 Durante la planificación

1. **Definir roles y estados antes de programar ahorra retrabajo.** Modelar primero la máquina de estados (disponible, reservada, entregada, vencida) permitió implementar y probar las reglas de negocio sin cambios de diseño posteriores.
2. **El alcance crece si no se acota.** Pasar de 2 a 3 roles aumentó un 50 % el esfuerzo del módulo. Conviene estimar con margen y priorizar lo que pide la rúbrica antes de añadir extras.
3. **Las restricciones del cliente definen la arquitectura.** No usar bases de datos ni servicios de pago llevó a un almacenamiento en servidor con persistencia en JSON. Es simple y sin costo, pero limita la escalabilidad y se registró como deuda técnica.
4. **Validar la usabilidad con el cliente desde el inicio.** La ruta guiada surgió en la revisión final. Una revisión temprana con el cliente habría permitido planificarla desde el principio.
5. **La interfaz no estaba estimada.** La UI/UX consumió 2.5 veces lo previsto. En el siguiente ciclo se estimará como una tarea propia.

### 5.2 Durante la implementación

6. **La seguridad desde el diseño reduce los hallazgos.** La validación estricta (listas blancas, sin campos desconocidos) y el frontend sin innerHTML hicieron que ZAP no encontrara vulnerabilidades de riesgo Medio o Alto.
7. **Separar la configuración del código desde el primer día.** SonarQube marcó como vulnerabilidad una contraseña de demostración escrita en el código. El problema no era el valor, sino dónde estaba.
8. **Los controles de seguridad afectan a las pruebas.** El limitador de peticiones bloqueó las pruebas E2E. Los parámetros de seguridad deben ser configurables por entorno y documentarse.
9. **Probar el pipeline pronto y en el sistema operativo real del runner.** Las diferencias entre Windows y Linux (rutas, permisos, UID) solo aparecieron en CI. Ejecutar los mismos scripts en local y en CI redujo esa brecha.
10. **Automatizar las herramientas de calidad como scripts versionados.** Tener sonar-scan.sh y zap-scan.sh en el repositorio permitió repetir los análisis antes y después de las correcciones y comparar métricas objetivamente.
11. **La cobertura no mide todo.** El frontend está fuera de la cobertura de Jest. Las pruebas E2E en navegador encontraron detalles de UX (plurales, alineación) que las pruebas unitarias no detectan.

## 6. Plan de mejora continua

El plan sigue el ciclo PDCA (Planificar, Hacer, Verificar, Actuar), con una revisión mensual de los indicadores a partir de las métricas del pipeline, de SonarQube y del registro de auditoría del sistema.

| # | Acción                                                             | Indicador (KPI)                                       | Meta                                                         | Plazo     |
|---|--------------------------------------------------------------------|-------------------------------------------------------|--------------------------------------------------------------|-----------|
| 1 | Automatizar pruebas E2E del frontend con Playwright en el pipeline | Flujos críticos cubiertos (8 identificados)           | 100 % de los flujos; 0 regresiones en producción             | 1 mes     |
| 2 | Endurecer el Quality Gate                                          | Calificación de seguridad en SonarQube; umbral de ZAP | Seguridad A; fallar el pipeline ante alertas de riesgo Medio | 2 semanas |

| # | Acción                                                                       | Indicador (KPI)                                           | Meta                                                 | Plazo         |
|---|------------------------------------------------------------------------------|-----------------------------------------------------------|------------------------------------------------------|---------------|
| 3 | Notificaciones por correo al aprobar, rechazar o asignar                     | Tiempo medio entre solicitud y decisión (auditoría)       | < 24 h                                               | 1 mes         |
| 4 | Reducir donaciones vencidas con alertas de caducidad próxima                 | Porcentaje de donaciones vencidas sin asignar             | < 5 % mensual                                        | 2 meses       |
| 5 | Migrar el almacenamiento a SQLite o PostgreSQL al superar el umbral de carga | Donaciones por mes; latencia p95                          | Migrar al superar 1 000 donaciones/mes; p95 < 300 ms | Según demanda |
| 6 | Observabilidad: logs estructurados y monitoreo de disponibilidad             | Disponibilidad mensual; tiempo de detección de incidentes | ≥ 99.5 %; < 5 min                                    | 2 meses       |
| 7 | Auditoría de accesibilidad WCAG 2.1 AA                                       | Puntuación de accesibilidad en Lighthouse                 | ≥ 95 en todas las vistas                             | 1 mes         |

## 6.1 Propuestas de innovación

- **IA para predecir donaciones futuras.** Entrenar un modelo de series temporales (por ejemplo, Prophet o regresión con estacionalidad) con el historial de donaciones por categoría, zona y temporada para anticipar la oferta y alertar a las organizaciones de escasez. KPI: error porcentual absoluto medio (MAPE) < 20 % a 4 semanas.
- **Emparejamiento inteligente donación–organización.** Un recomendador que sugiera al administrador la organización más adecuada según cercanía, categoría, capacidad declarada e historial de entregas. KPI: reducir un 30 % el tiempo de asignación y un 50 % las donaciones vencidas.
- **Optimización de rutas de recolección.** Agrupar donaciones cercanas para que una organización recoja varias en un solo recorrido. KPI: kilómetros por kilo entregado.
- **Confirmación de entrega con código QR y aplicación instalable (PWA).** Mejora la trazabilidad y permite operar con conectividad limitada. KPI: 100 % de las entregas confirmadas con evidencia.

## 6.2 Seguimiento

Cada acción tiene responsable, indicador y fecha de revisión. Los indicadores técnicos (cobertura, calificaciones de SonarQube y alertas de ZAP) se revisan automáticamente en cada ejecución del pipeline. Los de negocio (tiempos de asignación, vencidas y kilos entregados) salen de las métricas y de la auditoría del sistema. Las acciones que no alcancen su meta en dos revisiones consecutivas se replanifican en el siguiente ciclo PDCA.

## 7. Conclusiones

El proyecto cumplió los objetivos de las tres fases. El sistema es funcional, tiene autenticación y autorización robustas, 131 pruebas automatizadas con 99.84 % de cobertura y un pipeline CI/CD que despliega automáticamente en un entorno de prueba y bloquea cualquier cambio que no pase las pruebas, el análisis de calidad o el escaneo de seguridad. El esfuerzo superó el plan en +33 % por la ampliación del alcance a tres roles, la interfaz y la ruta guiada solicitada por el cliente, pero esa inversión dio como resultado un producto

más completo y seguro. El plan de mejora continua define los siguientes pasos con metas medibles, incluida la incorporación de inteligencia artificial para predecir donaciones.